

Enumerator

Gregorics Tibor and Pintér Balázs

Enumerator

1st Part

Collections and Enumerators

Gregorics Tibor and Pintér Balázs

Collection and its processing

- ❑ A **collection** is an object designed for storing items: you can place items in it, retrieve them, and remove them; the collection provides methods for these actions.

Examples: set, bag, sequence (which can specifically be a stack or queue if we limit its operations), array, tree, graph, or even a record.

- ❑ A collection can also be **virtual** when its elements don't need to be stored explicitly. For example, to represent a range of integers, it's sufficient to specify the endpoints of the interval.

- ❑ When we **process a collection** we actually process its elements.

- ❑ Examples:

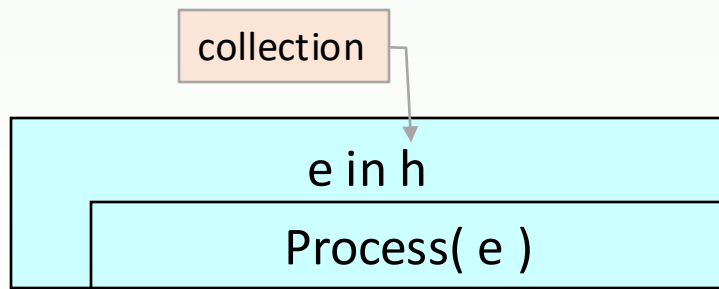
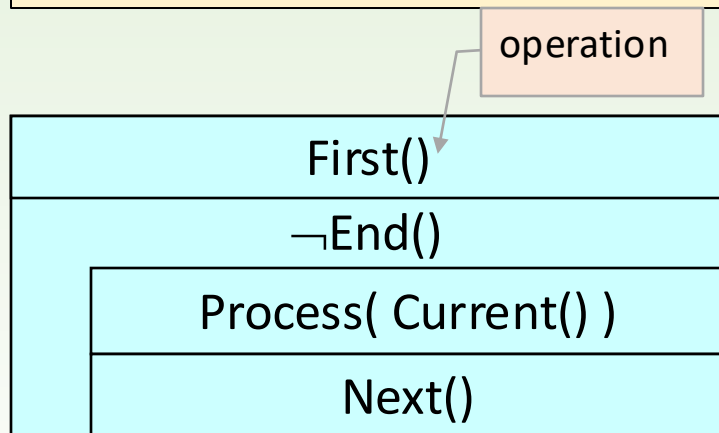
- Determine the largest element of a set based on a certain criterion.
- Let's count the negative numbers in a sequence of numbers.
- Enumerate the positive prime divisors of a natural number.

Enumeration

- ❑ To process the elements of a collection, we need to be able to **enumerate** (list) the items in the collection.
- ❑ We consider the enumeration of the elements of type E (denoted as $\text{enor}(E)$) as a sequence formed from the elements of the collection to be enumerated, on which the following **four operations** are applicable:
 - **First()**: positions onto the first element – it starts the enumeration.
 - **Next()** : positions onto the next element – continues the enumeration
 - $e := \text{Current()}$ ($e:E$): returns the next element of the enumeration
 - $l := \text{End()}$ ($l:\mathbb{L}$) : returns whether we are at the end of the enumeration
- ❑ The operations of the enumerations are only meaningful in certain situations (conditions); at other times, their effect is undefined.
 - We can distinguish three states of the list: **ready, in progress, completed**
 - **First()**: **ready**
 - **Next()** and **Current()**: **in progress**
 - **End()**: **in progress and completed**

Using an Enumerator

- Use the enumerator in an algorithm that ensures that the operations are always executed only in states where they are meaningful.



```
for( First(); !End(); Next() )  
{  
    Process(Current());  
}
```

Here, `h` is an enumerable object whose contents cannot be changed by the `foreach` loop. However, if there are objects in `h`, specifically their references, then their attributes can be modified.

```
for( var e : h )  
{  
    Process(e);  
}
```

the type of enumerated items can be automatically determined

Enumerator object

- ❑ If we resolve the enumeration of a collection's elements directly through the collection's methods, then we cannot initiate multiple enumerations on the same collection simultaneously.
- ❑ It is advisable for the enumeration to be performed by a separate so-called enumerator (iterator) object (Enor e) distinct from the collection, as multiple such objects can be created at the same time.
- ❑ An enumerator should:
 - Have the First(), Next(), Current(), End() operations
 - Should access the enumerated collection and use its operations
 - Should be instantiated by a method of the collection

Enumerator	
- ref : Collection	
+ First()	: void
+ Next()	: void
+ Current()	: int {query}
+ End()	: bool {query}

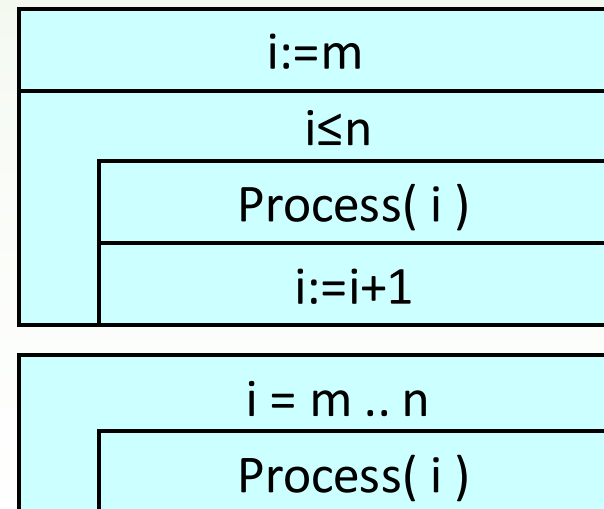
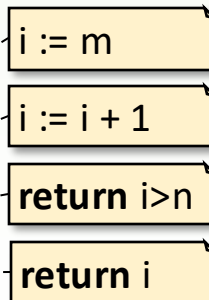
Enumerating an interval

Enumerating the integers that fall within the interval from the lower limit to the upper limit.

enor($\mathbb{Z}$)				
$\mathbb{Z}^*$	First()	Next()	$l := \text{End()}$ $l : \mathbb{L}$	$e := \text{Current()}$ $e : \mathbb{Z}$
$m, n : \mathbb{Z}$ $i : \mathbb{Z}$	$i := m$	$i := i + 1$	$l := i > n$	$e := i$

the boundaries of the interval determine the virtual collection

IntervalEnumerator	
- m : int	
- n : int	
- i : int	
+ First()	: void
+ Next()	: void
+ End()	: bool {query}
+ Current()	: int {query}

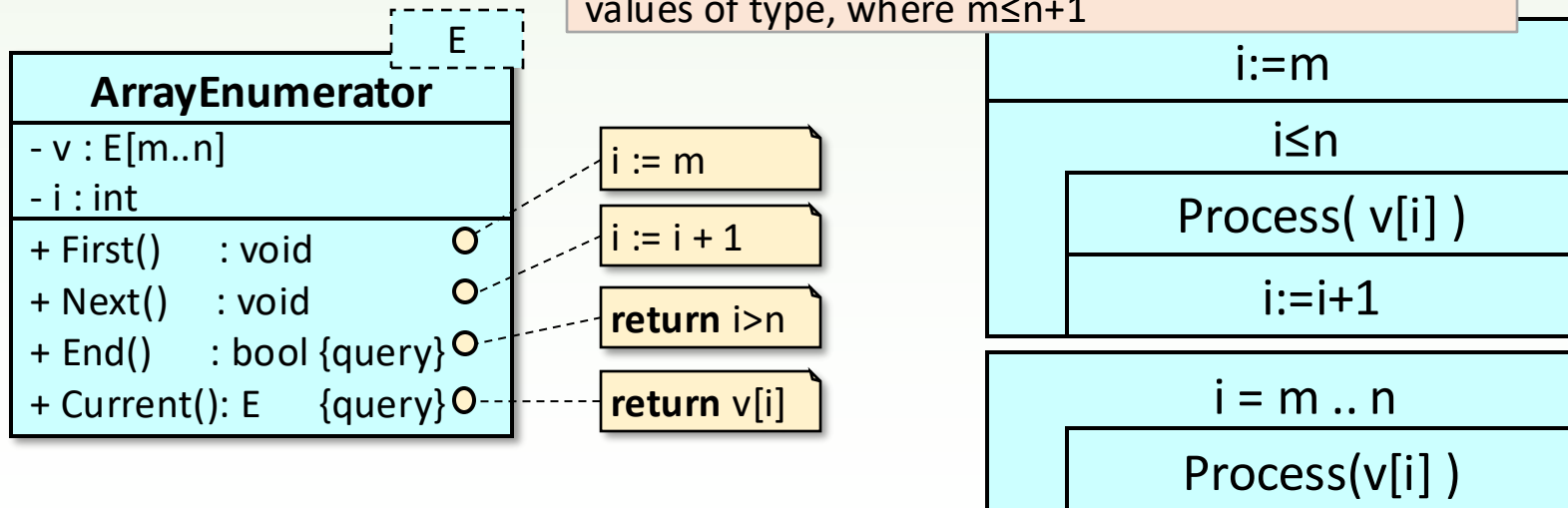


Enumerating a vector

Enumerating the elements of a one-dimensional array (vector) made up of values of type E from the beginning to the end.

enor(E)				
E^*	First()	Next()	$l := \text{End()}$ $l : \mathbb{L}$	$e := \text{Current()}$ $e : E$
$v : E^{m..n}$ $i : \mathbb{Z}$	$i := m$	$i := i + 1$	$l := i > n$	$e := v[i]$

One dimensional array indexed from m to n, with values of type, where $m \leq n + 1$

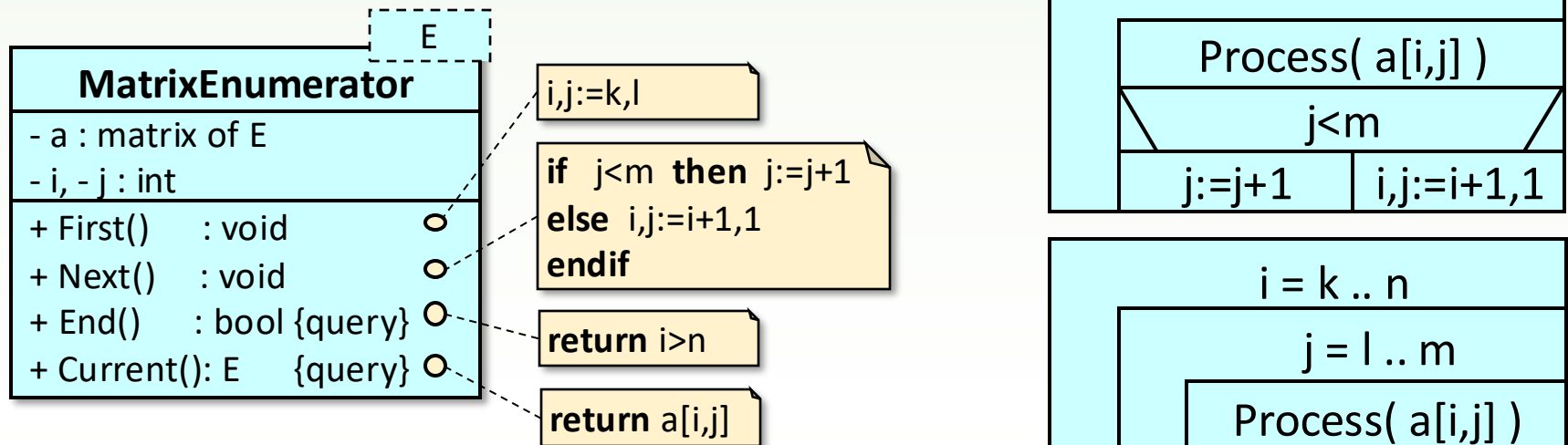


Enumerating a matrix

Enumerating a matrix of elements of type E row by row.

enor(E)				
E^*	First()	Next()	$l := \text{End()}$ $l: \mathbb{L}$	$e := \text{Current()}$ $e: E$
$a: E^{k..n \times l..m}$ $i, j: \mathbb{Z}$	$i, j := k, l$	if $j < m$ then $j := j + 1$ else $i, j := i + 1, l$	$l := i > n$	$e := a[i, j]$

The matrix has elements of type E. Rows are indexed from k to n, columns are indexed from l to m, and $k \leq n+1$, $l \leq m+1$



Enumerating a sequential infile

A sequential enumeration of a series of values. The series is exhausted by the end.

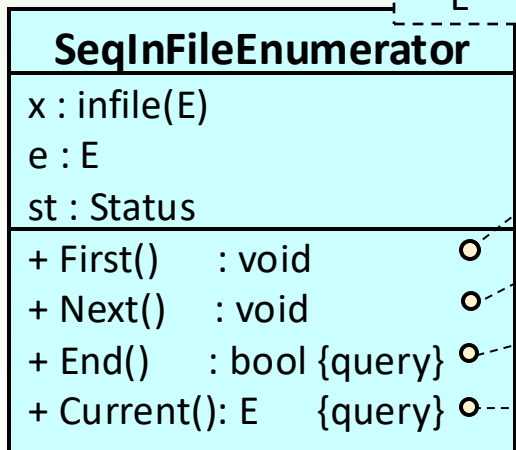
enor(E)				
E*	First()	Next()	l:= End() l:ℒ	o:= Current() o:E
x : infile(E) e : E st : Status	st,e,x:=read(x)	st,e,x:=read(x)	l:= st=abnorm	o:= e

Sequence of elements of type E where only the read() operation can be used:

st, e, x:=read(x)

if x = <> **then** st := abnorm

else st, e, x := norm, x₁, <x₂, ... , x_{|x|}>

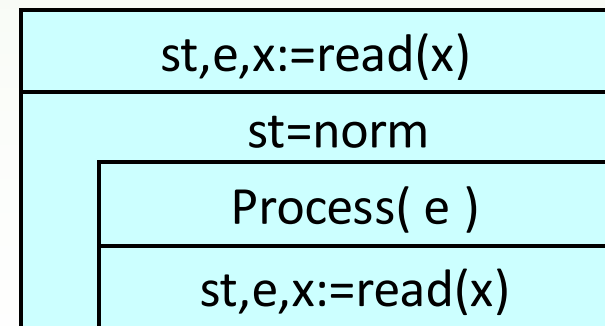


st,e,x:read

st,e,x:read

return st=abnorm

return e



prereading strategy

Enumerator

2nd part

Algorithmic patterns with enumerators

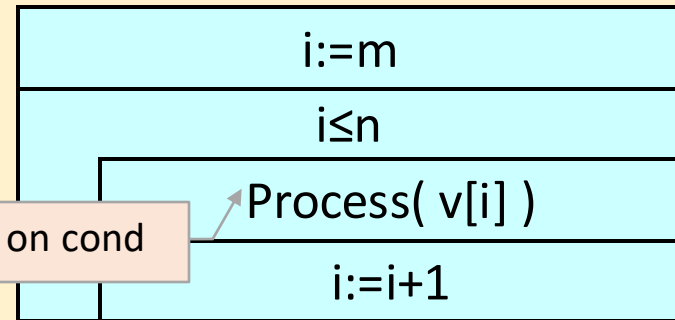
Gregorics Tibor and Pintér Balázs

From concrete to general APs

□ Algorithmic pattern for an array :

- $i:\mathbb{N}$ is the enumerator of $v:E^{m..n}$
- $\text{cond}:E \rightarrow \mathbb{L}$

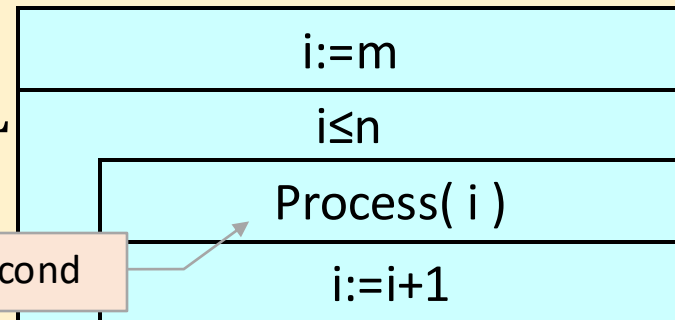
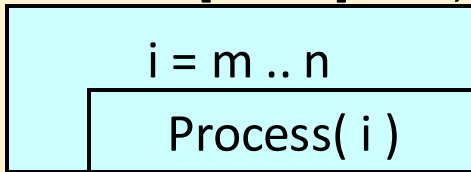
Depends on cond



□ Algorithmic pattern for a function in an interval:

- $i:\mathbb{N}$ is the enumerator of $[m .. n]$
- $f:[m .. n] \rightarrow H, \text{cond}:[m .. n] \rightarrow \mathbb{L}$

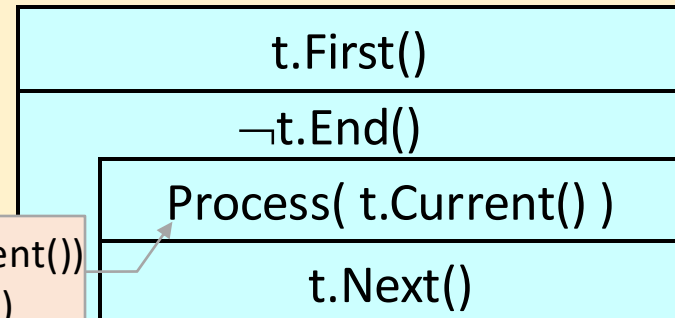
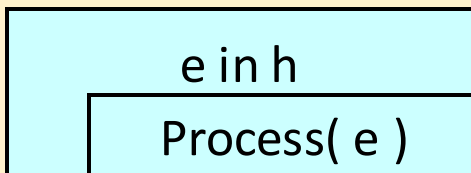
Depends on $f(i)$ and cond



□ Algorithmic pattern for an enumerator:

- $t:\text{enor}(E)$ enumerator
- $f:E \rightarrow H, \text{cond}:E \rightarrow \mathbb{L}$

Depends on $f(t.\text{Current}())$
and $\text{cond}(t.\text{Current}())$



Summation

Could be sum, product,
logical AND, OR,
concatenation of sequences

Summarize the values of the elements of an `enum(E)` type enumeration

$f : E \rightarrow H$
addition on elements of H :
 $+ : H \times H \rightarrow H$ $0 \in H$ *is the identity*

$S = (t: \text{enor}(E), s: H)$

$Q = (t = t')$

$R = (s = \sum_{e \in t'} f(e))$

$s = \sum_{i=1..|t'|} f(t'[i])$

Special case: conditional summation

$g : E \rightarrow H, \text{cond} : E \rightarrow \mathbb{L}$ $\sum_{\substack{e \in t' \\ \text{cond}(e)}} g(e)$

$$f(e) = \begin{cases} g(e) & \text{if } \text{cond}(e) \\ 0 & \text{otherwise} \end{cases}$$

`s := 0`

`t.First()`

`¬t.End()`

`s := s + f(t.Current())`

`t.Next()`

```
s = 0;
for ( var e : t )
{
    s += f(e);
}
```

`s = t.stream().reduce(0, Integer::sum)`

```
s = 0;
for ( var e : t )
{
    if (cond(e)) s += g(e);
}
```

`s = t.stream().filter(cond).reduce(0, Integer::sum)`

Counting

Count the elements with a given property of an enum(E) type

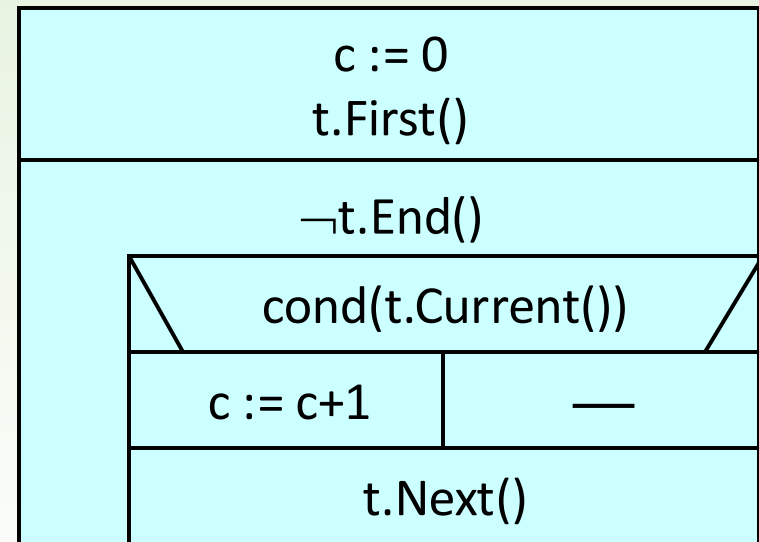
$\text{cond}: E \rightarrow \mathbb{L}$

$S = (t:\text{enor}(E), c:\mathbb{N})$

$Q = (t = t')$

$R = (c = \sum_{e \in t'} 1_{\text{cond}(e)})$

Counting is a special summation:

$$\sum_{e \in t'} f(e) \text{ azaz } f(e) = \begin{cases} 1 & \text{if } \text{cond}(e) \\ 0 & \text{otherwise} \end{cases}$$


```

c = 0;
for ( var e : t )
{
    if(cond(e)) ++c;
}
  
```

`c = t.stream().filter(cond).count();`

Selection (there's such an element)

Find the first element with the given property in an `enum(E)` type enumeration, provided we know it exists

$\text{cond}: E \rightarrow \mathbb{L}$

$S = (t:\text{enor}(E), \text{elem}:E)$

$Q = (t = t' \wedge \exists e \in t : \text{cond}(e))$

$R = ((\text{elem}, t) = \mathbf{SELECT}_{e \in t} \text{cond}(e))$

$\exists i \geq 1: \text{cond}(t'_i) \wedge \forall k \in [1..i-1] : \neg \text{cond}(t'_k) \\ \wedge \text{elem} = t'_i \wedge t = \langle t'_{i+1}, \dots, t'_{|t'|} \rangle$

`t.First()`

`¬cond(t.Current())`

`t.Next()`

`elem := t.Current()`

```
for ( var e : t )  
{  
  if ( cond(e) ){ elem = e; break; }  
}
```

`elem = t.stream().filter(cond).findFirst().get();`

The result cannot be null.

Linear search (we don't know)

Find the first element with the given property of an enum(E) type

cond: $E \rightarrow \mathbb{L}$

$S = (t:\text{enor}(E), l:\mathbb{L}, \text{elem}:E)$

$Q = (t = t')$

$R = ((l, \text{elem}, t) = \text{SEARCH}_{e \text{ in } t'} \text{ cond}(e))$

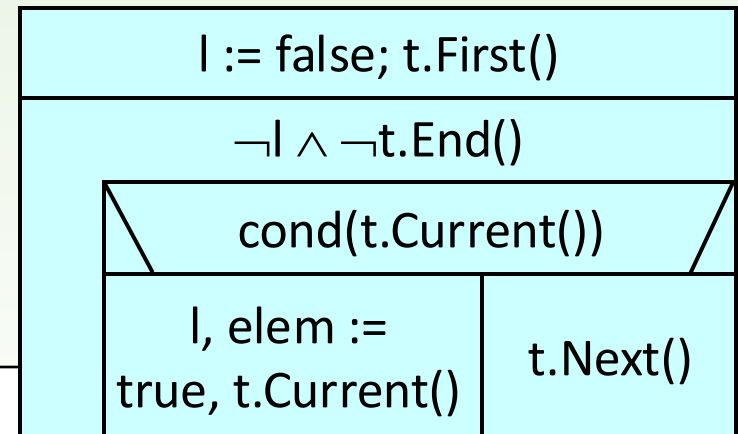
$(l = \exists i \in [1.. |t'|] : \text{cond}(t'_i)) \wedge$
 $(l \rightarrow i \in [1.. |t'|] \wedge \text{cond}(t'_i) \wedge \forall k \in [1..i-1] : \neg \text{cond}(t'_k)$
 $\wedge \text{elem} = t'_i \wedge t = \langle t'_{i+1}, \dots, t'_{|t'|} \rangle)$

```
elem = null
for ( var e : t )
{
    if ( cond(e) ) { elem = e; break; }
}
```

```
E find()
{
    for ( var e : t ) if ( cond(e) ) return e;
    return null;
}
```

Can be null

`elem = t.stream().filter(cond).findFirst().orElseThrow();`



Decision is a special case :

$l = \text{SEARCH}_{e \text{ in } t'} \text{ cond}(e)$ vagy $l = \exists_{e \in t'} \text{ cond}(e)$ `t.stream().anyMatch(cond);`

Optimistic linear search

Examine whether a given property holds true for every element of an enum(E) type. If not, let's identify the first element for which it does not.

$\text{cond}: E \rightarrow \mathbb{L}$

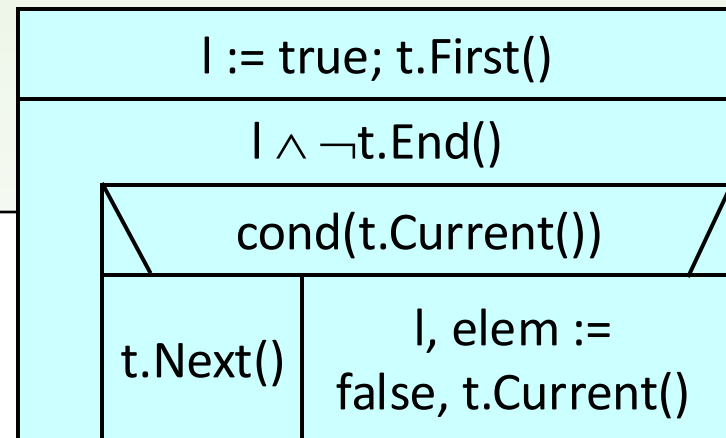
$S = (t:\text{enor}(E), l:\mathbb{L}, \text{elem}:E)$

$R = (t = t')$

$Q = ((l, \text{elem}, t) = \forall \text{SEARCH}_{e \text{ in } t'} \text{ cond}(e))$

$(l = \forall i \in [1..|t'|] : \text{cond}(t'_i)) \wedge$
 $(\neg l \rightarrow i \in [1..|t'|] \wedge \neg \text{cond}(t'_i) \wedge \forall k \in [1..i-1] : \text{cond}(t'_k)$
 $\wedge \text{elem} = t'_i \text{ and } t = \langle t'_{i+1}, \dots, t'_{|t'|} \rangle)$

```
elem = null;
for ( var e : t )
{
    if ( cond(e) ) { elem = e; break; }
}
```



We mainly use it for decisions:

$l = \forall \text{SEARCH}_{e \text{ in } t'} \text{ cond}(e)$ vagy $l = \forall e \in t' \text{ cond}(e)$

`t.stream().allMatch(cond);`

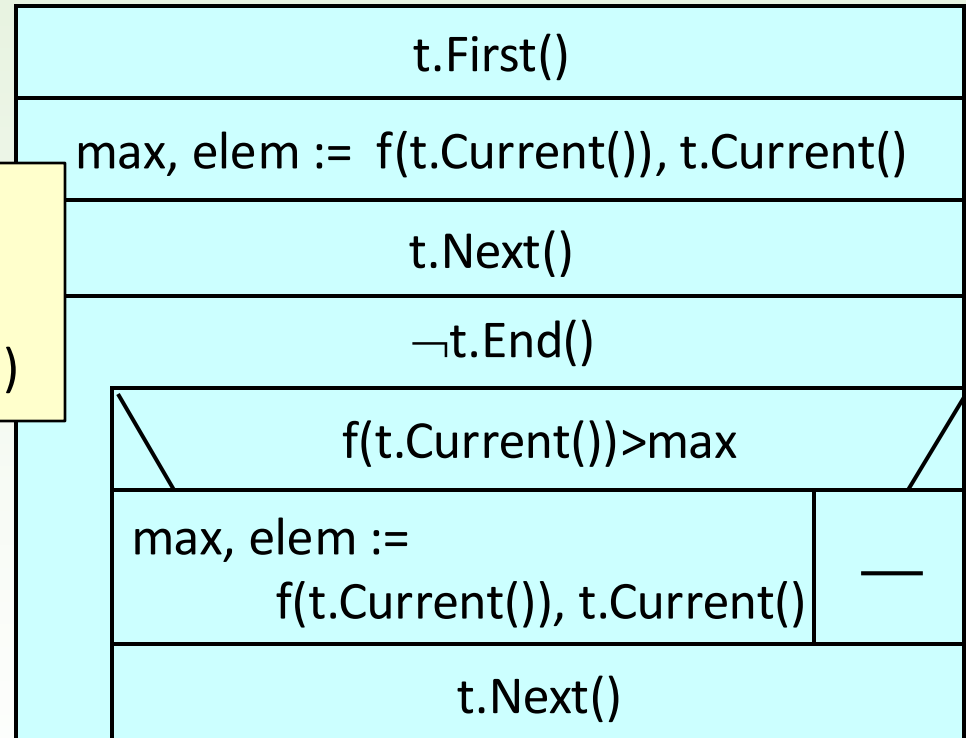
Maximum selection

Determine one of the largest elements of an enumerator (E) type based on a given criterion, along with its value.

$f : E \rightarrow H$
H is sortable

$S = (t:enor(E), elem:E, max:H)$
 $Q = (t = t' \wedge |t| > 0)$
 $R = ((max, elem) = \mathbf{MAX}_{e \in t'} f(e))$

$\exists i \in [1..|t'|] : \forall k \in [1..|t'|] :$
 $f(t'_k) \leq f(t'_i)$
 $\wedge elem = t'_i \wedge max = f(elem)$



- we can use MIN (<) too, not just MAX (>)
- elem *can be omitted*, max *cannot be*

elem = t.stream().max(comparator);

Conditional maximum search

Find among the elements with a given property of an enumerator (E) type the largest element based on a given criterion

$f : E \rightarrow H$
 $\text{cond} : E \rightarrow \mathbb{L}$
 H is sortable

$S = (t : \text{enor}(E), l : \mathbb{L}, \text{elem} : E, \text{max} : H)$
 $Q = (t = t')$
 $R = (l, \text{max}, \text{elem}) = \text{MAX}_{\substack{e \text{ in } t' \\ \text{felt}(e)}} f(e)$

- can use MIN (<), not just MAX (>)
 - elem can be omitted, max not

```
bool l = false;
for ( var e : h )
{
    if (!cond(e)) continue;
    v = f(e);
    if ( !l ) { l = true; max = v; elem = e; }
    else if ( v > max ) { max = v; elem = e; }
}
```

$(l = \exists i \in [1..|t'|] : \text{felt}(t'_i)) \wedge$
 $(l \rightarrow i \in [1..|t'|] \wedge \text{felt}(t'_i) \wedge$
 $\wedge \forall k \in [1..|t'|] : (\text{cond}(t'_k) \rightarrow f(t'_k) \leq f(t'_i))$
 $\wedge \text{elem} = t'_i \wedge \text{max} = f(\text{elem}))$

$l := \text{false}$
 $t.\text{First}()$

$\neg t.\text{End}()$

$\text{cond}(t.\text{Current}())$

$\neg l \vee f(t.\text{Current}()) > \text{max}$

$l, \text{max}, \text{elem} :=$
 $\text{true}, f(t.\text{Current}()), t.\text{Current}()$

$t.\text{Next}()$

$v = t.\text{stream}.\text{filter}(\text{cond}).\text{max}();$

Enumerator

3rd part

Analogy and testing

Gregorics Tibor and Pintér Balázs

Analogy

1. We anticipate the algorithmic pattern that solves the problem.
2. We specify the task with a **postcondition** similar to that of the algorithmic pattern.
3. We record the differences between the task and the algorithm template.
 - the type of the **enumerator** and the **enumerated elements**
 - the concrete **functions** ($f : E \rightarrow H$, $felt : E \rightarrow \mathbb{L}$)
 - the **operation** of set H (if applicable)
 - example: $(H, >) \Rightarrow (\mathbb{Z}, >)$ or $(\mathbb{Z}, <)$
 - example: $(H, +) \Rightarrow (\mathbb{Z}, +)$ or $(\mathbb{R}, *)$ or $(\mathbb{L}, \wedge)$
 - the **renaming of variables**
4. We incorporate the above differences into the algorithm template, so we obtain the algorithm that solves the problem.

Theorem:

If a problem and the algorithm pattern can be matched to each other via analogy, then the correspondingly adapted version of the algorithm from the pattern will solve the given problem.

Testing strategies

- ❑ **Black box:** test cases are created based on the problem (specification).
 - test cases written with input data that either satisfy the precondition (valid) or violate it (invalid).
 - test cases generated based on the postcondition
 - ...

- ❑ **White box:** test cases created based on the code.
 - Testing every instruction of the algorithm.
 - Testing every control structure (branch, loop) of the algorithm.
 - ...

- ❑ **Grey box:** Test cases that verify the operation of the algorithm generated by the executable specification.
 - If the executable specification relies on algorithm patterns, then the typical test cases of the given algorithm pattern must be examined.

Test cases for algorithmic patterns

□ According to enumeration (for each algorithm pattern)

- Enumerations of varying **lengths**: we will test with zero, one, and longer lengths.
- Whether we process the **first** or the **last** element of the enumeration.

Counting: only the beginning and the end have a given property
Searching: only the beginning or the end has a given property
Maximum selection: the beginning or the end is the largest

□ By function

- summation: **identity element**, **scaling** with the length
- search, counting: There **is or isn't** an element with the property.
- max. sel.: a **single**, or **several** identical maximum values
- cond. max. sea.:
 - **has or doesn't have** element with the property
 - **one or more** identical maximum values
 - **cond is not true for maximum value**
- Testing the expressions $\text{cond}(e)$ and $f(e)$ (properties of the operations used in the expressions, their domain).

Example

We've measured the elevation above sea level at various points along a line on the Earth's surface, and we've stored these measurements in a sequence. What's the depth of the deepest depression in the surface, and where is it located?

$$S = (x : \mathbb{R}^*, l : \mathbb{L}, \text{max} : \mathbb{R}, \text{where} : \mathbb{N})$$

$$Q = (x = x')$$

$$R = (Q \wedge (l, \text{max}, \text{where}) = \mathbf{MAX}_{i=2..|x|-1} x[i] \wedge x[i-1] > x[i] < x[i+1])$$

Conditional maximum search:

$t:\text{enor}(E) \sim i=2..|x|-1$

$f(e) \sim x[i]$

$\text{felt}(e) \sim x[i-1] > x[i] < x[i+1]$

$H, > \sim \mathbb{R}, >$

$$S = (f : \text{infile}(\text{Triple}), l : \mathbb{L}, \text{max} : \mathbb{R}, \text{where} : \mathbb{N})$$

Triple = rec(pre:ℝ, mid:ℝ, post:ℝ, ind:ℕ)

$$Q = (f = f')$$

$$R = ((l, \text{max}, \text{elem}) = \mathbf{MAX}_{e \text{ in } f'} e.\text{mid}) \wedge e.\text{pre} > e.\text{mid} < e.\text{post}$$

$$\wedge l \rightarrow \text{where} = \text{elem.ind}$$

Conditional maximum search:

$t:\text{enor}(E) \sim \text{st}, e, f := \text{read}(f)$

$f(e) \sim e.\text{mid}$

$\text{felt}(e) \sim e.\text{pre} > e.\text{mid} < e.\text{post}$

$H, > \sim \mathbb{R}, >$

Infile

read() : bool×Triple

Model

$S = (f : \text{infile}(\text{Triple}), l : \mathbb{L}, \text{max} : \mathbb{R}, \text{where} : \mathbb{N})$

$\text{Triple} = \text{rec}(\text{pre} : \mathbb{R}, \text{mid} : \mathbb{R}, \text{post} : \mathbb{R}, \text{ind} : \mathbb{N})$

$Q = (f = f')$

$R = ((l, \text{max}, \text{elem}) = \mathbf{MAX}_{e \text{ in } f'} e.\text{mid}) \wedge$

$e.\text{pre} > e.\text{mid} < e.\text{post}$

$\wedge l \rightarrow \text{where} = \text{elem}.\text{ind})$

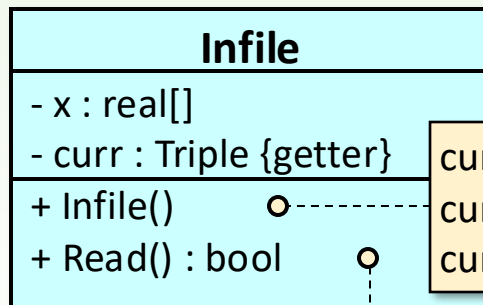
Conditional maximum search:

$t:\text{enor}(E) \sim \text{st}, e, f := \text{read}(f)$

$f(e) \sim e.\text{mid}$

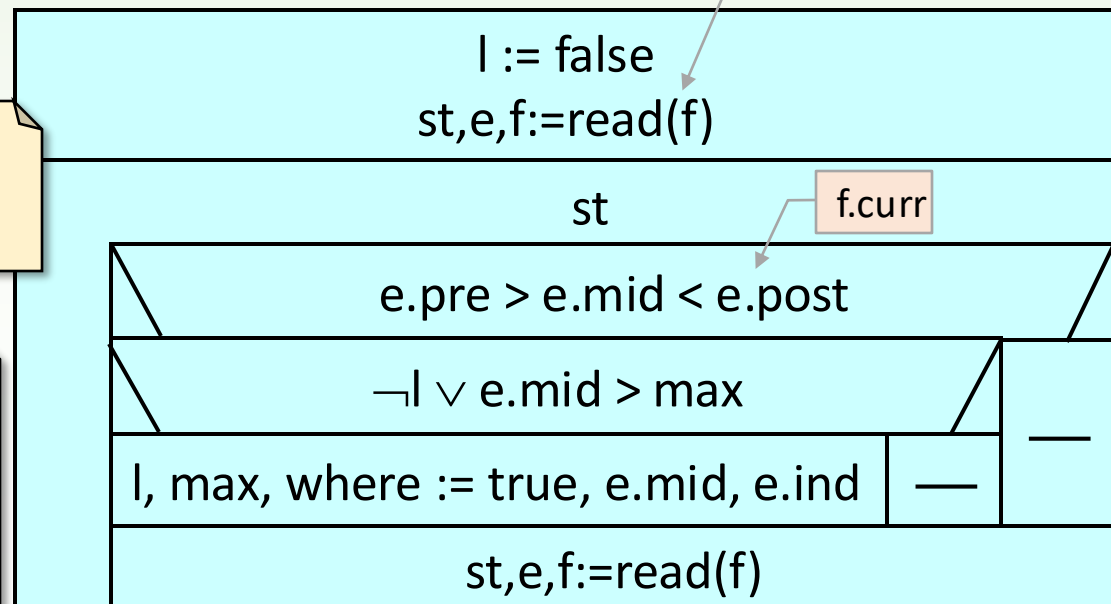
$\text{felt}(e) \sim e.\text{pre} > e.\text{mid} < e.\text{post}$

$H, > \sim \mathbb{R}, >$



curr.mid := x[1]
curr.post := x[2]
curr.ind := 1

if curr.ind+2 > |x| **return false** **endif**
curr.pre := curr.mid
curr.mid := curr.post
curr.post := x[ind+2]
curr.ind := curr.ind + 1
return true



Sequential input file

```
public static List<Double> fillInFromFile(String filename)
{
    List<Double> x = new ArrayList<>();

    try (Scanner sc = new Scanner(new File(filename)))
    {
        while (sc.hasNextDouble())
            x.add(sc.nextDouble());
    }
    catch (FileNotFoundException e)
    {
        System.out.println("File open error.");
    }
    return x;
}
```

Conditional maximum search

```
public static double max;
public static int ind;

public static boolean maxSearch(List<Double> x){
    boolean l = false; max = 0.0; ind = 0;
    for(int i = 1; i < x.size() - 1; ++i){
        if (!(x.get(i - 1) > x.get(i) && x.get(i) < x.get(i + 1))) continue;
        if (l){
            if (max < x.get(i)) {
                max = x.get(i); ind = i;
            }
        } else {
            l = true; max = x.get(i); ind = i;
        }
    }
    return l;
}
```

```
public static void main(String[] args) {
    List<Double> x = fillInFromFile("input.txt");
    if (maxSearch(x)){
        double max = Depression.max;
        int ind = Depression.ind;
        System.out.printf("Height of the highest depression: %.2f meter on the %d-th position\n",
max, ind);
    } else {
        System.out.println("There is no depression.");
    }
}
```

Testing

Test cases for conditional maximum search			
By enumeration	length: 0	$x = \langle \rangle, \langle 2.0, 1.0 \rangle$	$\rightarrow l = \text{false}$
	length: 1	$x = \langle 2.1, 1.0, 2.4 \rangle$	$\rightarrow l = \text{true}, \text{max} = 1.0, \text{ind} = 2$
	length: több	$x = \langle 1.0, 2.0, 1.5, 4.0, 2.0 \rangle$	$\rightarrow l = \text{true}, \text{max} = 1.5, \text{ind} = 3$
	first part	$x = \langle 3.0, 2.5, 3.0, 2.0, 3.0 \rangle$	$\rightarrow l = \text{true}, \text{max} = 2.5, \text{ind} = 2$
	last part	$x = \langle 3.0, 2.0, 3.0, 2.5, 3.0 \rangle$	$\rightarrow l = \text{true}, \text{max} = 2.5, \text{ind} = 4$
By algorithm	Cond all false	$x = \langle 1.0, 2.0, 3.0, 4.0, 5.0 \rangle$	$\rightarrow l = \text{false}$
	Cond true	$x = \langle 1.0, 2.0, 1.0, 4.0, 2.0 \rangle$	$\rightarrow l = \text{true}, \text{max} = 1.0, \text{ind} = 3$
	one maximum	$x = \langle 3.0, 1.5, 3.0, 2.5, 3.0 \rangle$	$\rightarrow l = \text{true}, \text{max} = 2.5, \text{ind} = 4$
	more maximum	$x = \langle 3.0, 2.0, 3.0, 2.0, 3.0 \rangle$	$\rightarrow l = \text{true}, \text{max} = 2.0, \text{ind} = 2,4$

Test environment

To test the methods of a project, ...

```
import org.junit.Test;
import org.junit.Assert;
import java.util.List;
import java.util.Arrays;
import java.util.ArrayList;
import depression.Depression;
```

```
public class CondMaxSearchTests
{
```

```
    @Test
```

```
    public void testNullLengthEnumerator() { ... }
```

```
    @Test
```

```
    public void Test_OneElement() { ... }
```

```
    @Test
```

```
    public void testNoDepression() { ... }
```

```
    @Test
```

```
    public void testSeveralMaxs () { ... }
```

```
}
```

← Marks a method as unit test, JUnit runs it automatically

1. Create a new project in the same solution as previous one (Depression): [Add new project](#)
2. The new project should be [TestProject](#)
3. [Add/Project Reference](#) : Depression
4. Create TestMethods

CondMaxSearchTests.java

Unit Test Cases

```
@Test
public void testNullLengthEnumerator() {
    List<Double> x = new ArrayList<>();
    boolean l = Depression.maxSearch(x);
    Assert.assertEquals(false, l);

    x.add(1.0); x.add(2.0);
    l = Depression.maxSearch(x);
    Assert.assertEquals(false, l);
}
```

```
@Test
public void testNoDepression() {
    List<Double> x = Arrays.asList(1.0, 2.0, 3.0, 4.0, 5.0);
    boolean l = Depression.maxSearch(x);
    Assert.assertEquals(false, l);
}
```

...

CondMaxSearchTests.java

Unit Test Cases

```
...
@Test
public void testOneLengthEnumerator() {
    List<Double> x = Arrays.asList(2.1, 1.0, 2.4);
    boolean l = Depression.maxSearch(x);
    Assert.assertEquals(true, l);
    Assert.assertEquals(1.0, Depression.max, 0.001);
    Assert.assertEquals(1, Depression.ind);
}

@Test
public void testSeveralMaxs() {
    List<Double> x = Arrays.asList(3.0, 2.0, 3.0, 2.0, 3.0);
    boolean l = Depression.maxSearch(x);
    Assert.assertEquals(true, l);
    Assert.assertEquals(2.0, Depression.max, 0.001);
    Assert.assertEquals(true, Depression.ind == 1 || Depression.ind == 3);
}

@Test
public void testNoTextFile() {
    List<Double> result = Depression.fillInFromFile("non_existing_file.txt");
    Assert.assertEquals(true, result.isEmpty());
}
```

CondMaxSearchTests.java